

SC1007 Data Structures & Algorithms

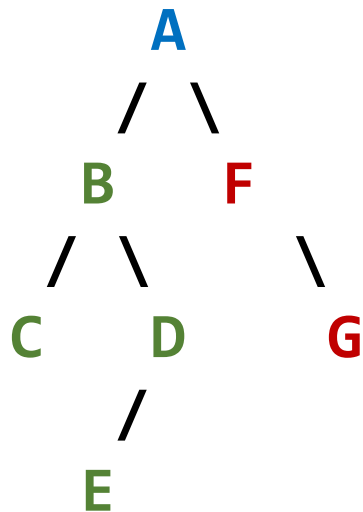
Tutorial 3. Binary Trees & Binary Search Trees

Q1. Constructing a BT from Pre-order & In-order Traversal

You are given a pre-order traversal and an in-order traversal of a binary tree. Draw the binary tree from the two traversal results.

- **pre-order:** A, B, C, D, E, F, G
- **In-order:** C, B, E, D, A, F, G

- The first visiting node of the pre-order traversal is the root. From the in-order traversal, you can find that **C, B, E, D** is in the left subtree and **F, G** is in the right subtree. The second visiting node of the pre-order traversal is the root of the left subtree if the left subtree is not empty. Otherwise, it will be the root of right subtree. From the in-order traversal, you can find that **C** is in the left subtree and **E, D** is in the right subtree. etc. The binary tree is



pre-order: **A**, B, C, D, E, F, G

In-order: **C**, B, **E**, **D**, A, **F**, **G**

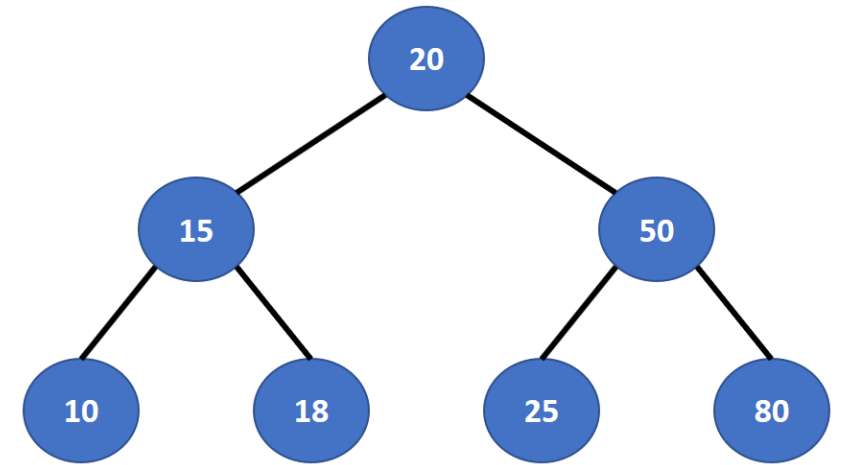
Q2. Level Order Traversal

Write an iterative function `levelOrderTraversal` that **prints a level-by-level traversal of a binary search tree using a queue, starting at the root node level.**

Note that you should only use **`enqueue()`** or **`dequeue()`** operations when you add or remove integers from the queue. Remember to empty the queue at the beginning, if the queue is not empty.

Motivation

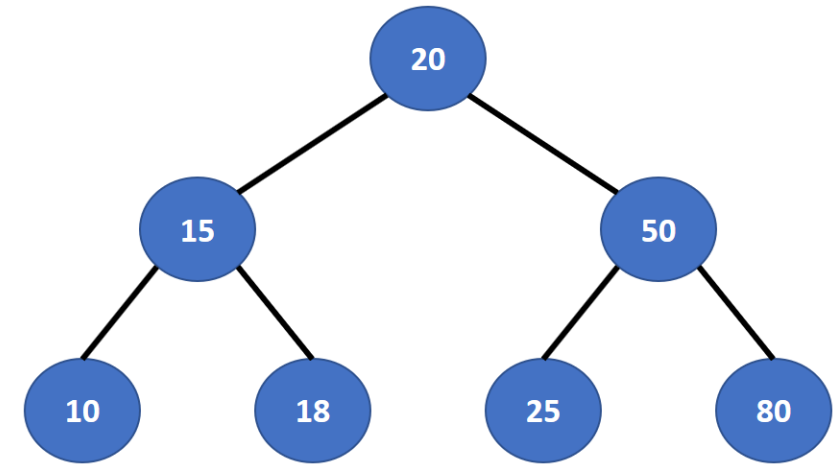
- A level-order traversal of a BST is like taking a line-by-line reading of the tree, from top to bottom and left to right, just as you would read text on a page
- This traversal method ensures you visit all nodes at the current depth before moving on to the next level
- Why a queue? Because a queue follows a first-in, first-out (FIFO) principle, which is perfect for keeping track of which node to visit next



Level-order Tree Traversal:
20 15 50 10 18 25 80

Algorithm

- **Initialization:** You begin with the root node of the tree and place this node in a queue
- **Dequeue and Visit:** Remove the front node from the queue (starting with the root node), and consider this a "visit" to the node
- **Enqueue the Children:** After visiting a node, you take its children (left then right) and put them at the end of the queue → This ensures they will be visited in the order they were added
- **Repeat the Process:** Keep removing the front node from the queue, visiting it, and adding its children to the back of the queue until the queue is empty



Level-order Tree Traversal:
20 15 50 10 18 25 80

```

1  def levelOrderIterative(root):
2
3
4      q = Queue()
5
6      temp = root
7
8      if temp is not None:
9          enqueue(q, temp)
10         while not is_empty(q.head):
11             temp = dequeue(q)
12             print(temp.item, end=' ')
13             if temp.left is not None:
14                 enqueue(q, temp.left)
15             if temp.right is not None:
16                 enqueue(q, temp.right)
17         }
18     }
19 }

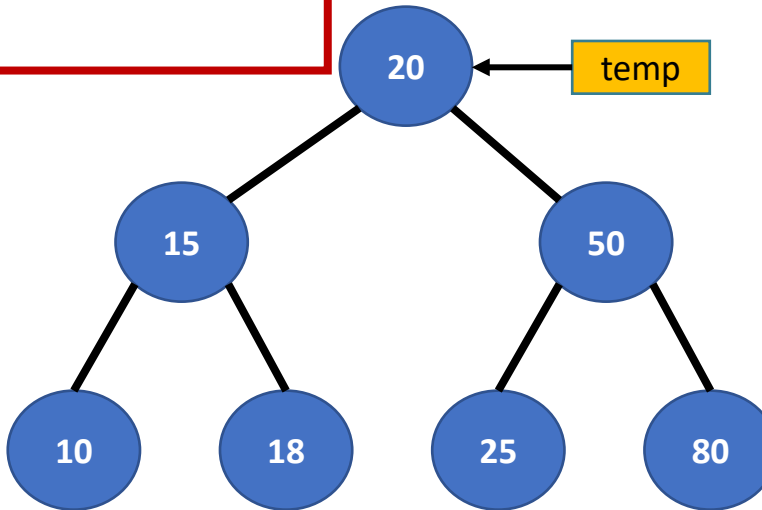
```



queue

15	10	25	80
----	----	----	----

Print: 20 15 50 10 18 25 80



Q3. Pre-order Traversal

Write an iterative function **preOrderIterative()** that **prints the pre-order traversal of a binary search tree using a stack.**

Note that you should only use **push()** or **pop()** operations when you add or remove integers from the stack.

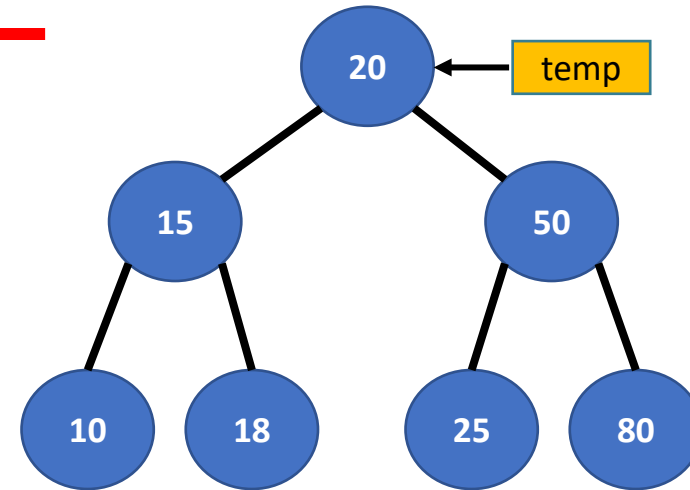
Algorithm

- 1) Create an empty stack nodeStack and push the root node to the stack
- 2) Do following while nodeStack is not empty
 - a) **Pop** an item from stack and print it
 - b) **Push** right child of popped item to stack
 - c) **Push** left child of popped item to stack

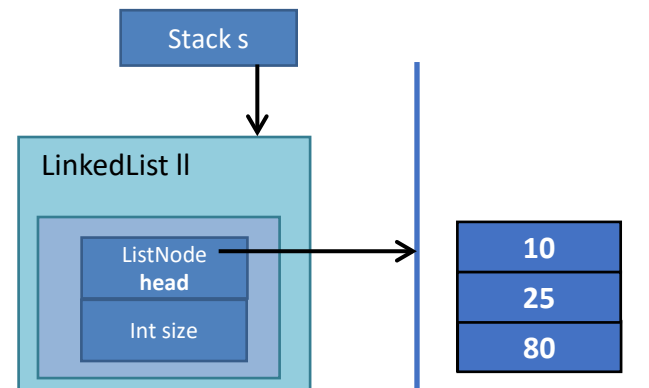
*Right child is pushed **before** left child to make sure that left subtree is processed first*

```

1  def preOrderIterative(root):
2
3      s = Stack()
4
5      temp = root
6
7      if temp is None:
8          return
9      push(s, temp)
10
11     while not isEmpty(s):
12         temp = pop(s)
13         print(temp.item, end=' ')
14
15         if temp.right is not None:
16             push(temp.right)
17         if temp.left is not None:
18             push(temp.left)
19
20 }
```

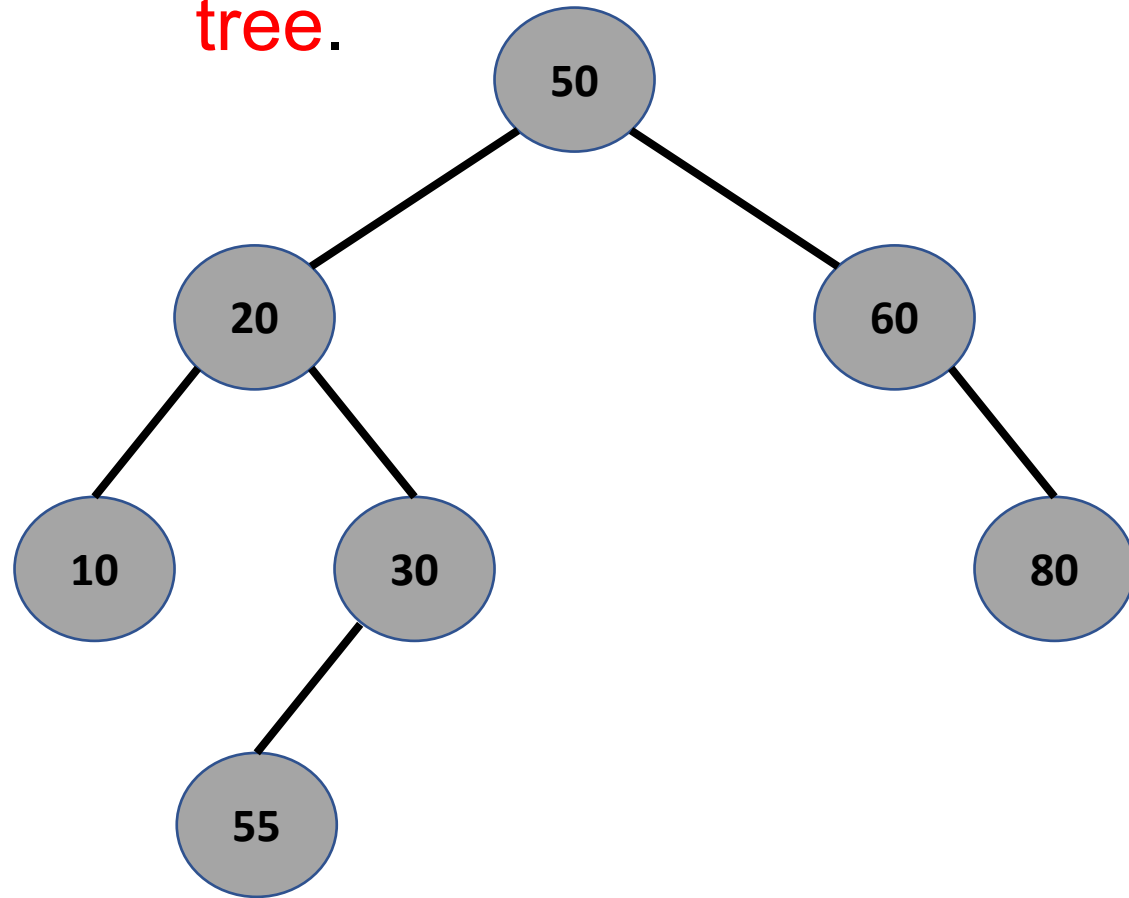


Print: 20 15 10 18 50 25 80



Q4. Finding the Maximum Depth

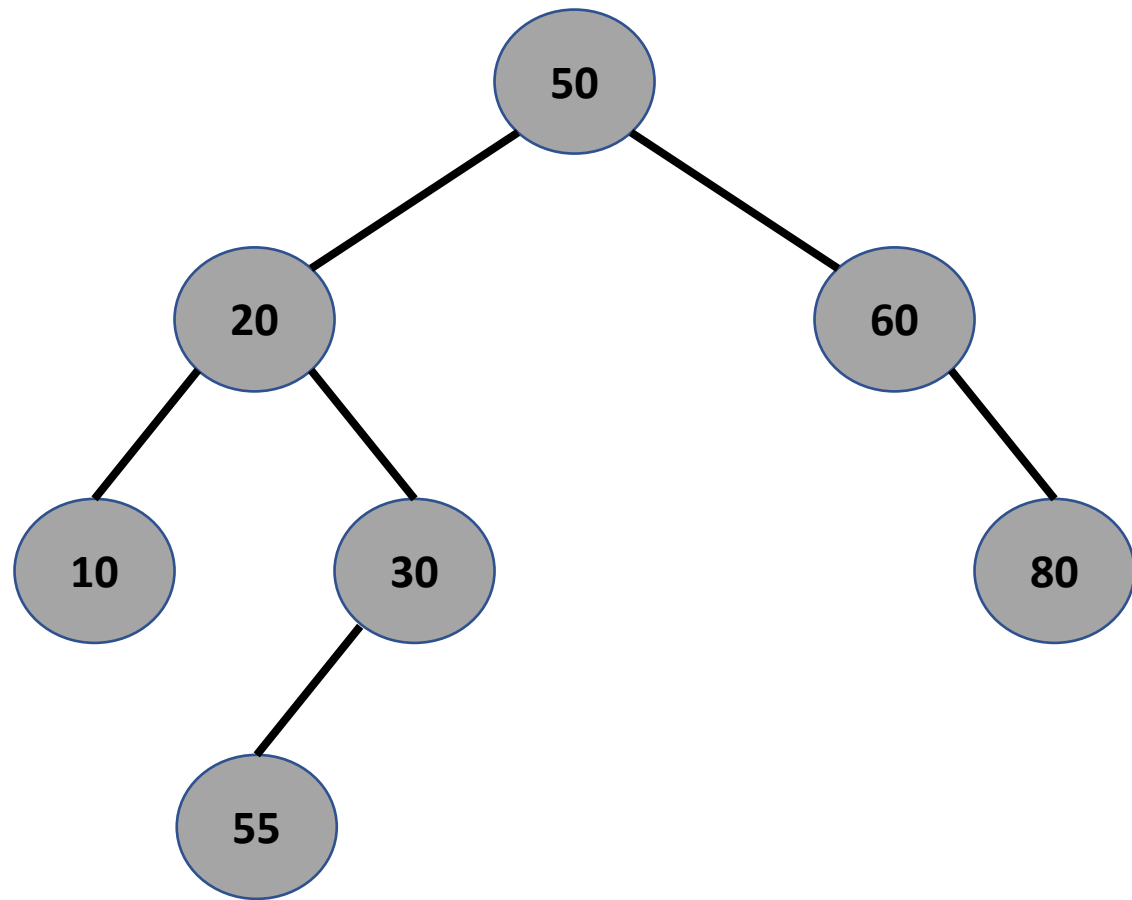
Write a Python function to **find the maximum depth of a binary tree**.



Max-Depth: **3**

```
def maxDepth(node):
```

Finding the maximum depth of a binary tree means measuring the longest path from the root node down to the farthest leaf node.



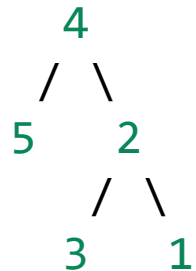
```
1  def maxDepth(node):  
2      if node is None:  
3          return -1  
4      else:  
5          ldepth = maxDepth(node.left)  
6          rdepth = maxDepth(node.right)  
7  
8          if ldepth > rdepth:  
9              return ldepth+1  
10         else:  
11             return rdepth+1  
12  
13
```

```

1  def maxDepth(node):
2      if node is None:
3          return -1
4      else:
5          ldepth = maxDepth(node.left)
6          rdepth = maxDepth(node.right)
7
8          if ldepth > rdepth:
9              return ldepth+1
10         else:
11             return rdepth+1
12
13

```

Example



1. Start at root **4**.
2. **ldepth** = **maxDepth(5)** → since **5** is a leaf, left and right children return **-1**, so **ldepth** = **max(-1, -1)+1 = 0**.
3. **rdepth** = **maxDepth(2)** → recursively compute depth for 2's children:
 - **maxDepth(3)** → **leaf** → returns 0
 - **maxDepth(1)** → **leaf** → returns 0
 - So **rdepth** = **max(0,0)+1 = 1**
4. For root **4**, **max(ldepth, rdepth)+1 = max(0,1)+1 = 2**

The maximum depth of the tree is **2 edges** (or 3 nodes if counting nodes).